



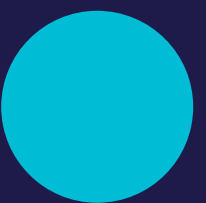
DASTURLASH 1 • DAS1110

4-Ma'ruza: Operatorlar

Shart operatorlari (if, if-else, switch-case). Takrorlash operatorlari (for, while, do-while).

O'zbekiston Milliy universiteti Jizzax filiali | Amaliy matematika fakulteti

O'qituvchi: Toshpolatov Muxiddin, PhD, dotsent | 2025/2026-o'quv yili



O'tgan darsni eslaymiz

- Amal (operator) — operandlar ustida harakat bajaruvchi belgi; unar va binar turlarga bo'linadi.
- Arifmetik, solishtirish va mantiqiy amallarni o'rgandik: +, -, *, /, %, ==, !=, &&, ||.
- Shartli (ternary) amal: shart ? qiymat1 : qiymat2;
- Amallar ustuvorligi — qavslar natijani qanday o'zgartirishini ko'rdik.
- Bugun esa dasturga QAROR QABUL QILISH (shart) va TAKRORLASH (sikl) qobiliyatini qo'shamiz.

Bugungi darsning rejasi

- I-BO'LIM: Oddiy/qo'shma operator va shart operatorlari
 - if, if-else, ko'p shartli zanjir, (bonus: switch-case)
- II-BO'LIM: Takrorlash operatori — for
 - for sikli sintaksisi va ishlash tartibi
- III-BO'LIM: Takrorlash operatorlari — while, do-while
 - while, do-while, break/continue, qaysi siklni qachon tanlash

Dars yakunida siz nimalarga erishasiz?

- Oddiy va qo'shma (blok) operator orasidagi farqni tushuntira olasiz.
- if va if-else operatorlaridan foydalanib, dasturga shartli qaror qabul qildirira olasiz.
- for, while va do-while sikllarining har birini to'g'ri yoza olasiz va ular orasidagi farqni bilasiz.
- break va continue kalit so'zlaridan foydalanib, sikl ishlashini boshqara olasiz.
- Cheksiz sikl (infinite loop) xatosidan qanday saqlanishni bilasiz.



1-BO'LIM

Oddiy/qo'shma operator va shart operatorlari

Dasturga birinchi marta QAROR QABUL QILISH qobiliyatini qo'shamiz.

Oddiy va qo'shma operator (statement)

- OPERATOR (statement) — dasturda bajariladigan bitta to'liq buyruq, odatda ";" bilan tugaydi.
- ODDIY operator — bitta yagona buyruq: `Console.WriteLine("Salom");`
- QO'SHMA operator (blok) — figurali qavslar { } ichiga olingan BIR NECHTA operatorlar to'plami, BITTA butunlik sifatida qaraladi.
- Shart va sikl operatorlari ko'pincha o'zidan keyin QO'SHMA operator (blok)ni bajaradi — shuning uchun { } qanday ishlashini tushunish juda muhim.

if operatori

- if operatori — biror shart TRUE bo'lgandagina ma'lum kodni bajarish imkonini beradi.
- Sintaksisi: `if (shart) { // shart true bo'lsa shu yerdagi kod bajariladi }`
- Shart — natijasi bool (true/false) bo'lgan har qanday ifoda bo'lishi mumkin (masalan `yosh >= 18`).
- Agar blok ichida FAQAT bitta operator bo'lsa, `{ }` qavslarni yozmasa ham bo'ladi — lekin ANIQLIK uchun har doim yozish tavsiya etiladi.

Misol: oddiy if operatori

```
int yosh = 20;

if (yosh >= 18)
{
    Console.WriteLine("Siz kattasiz.");
}
```

► NATIJA (Console chiqishi):

Siz kattasiz.

if-else operatori

- if-else — shart TRUE bo'lganda BIR xil kod, FALSE bo'lganda BOSHQA kod bajarilishini ta'minlaydi.
- Sintaksisi: `if (shart) { ... } else { ... }`
- Bir nechta shartni ketma-ket tekshirish uchun `else if` zanjiridan foydalaniladi.
- Faqat BITTA blok (if, yoki mos else if, yoki eng oxirgi else) bajariladi — qolganlari o'tkazib yuboriladi.

Misol: else if zanjiri

```
int ball = 75;

if (ball >= 90)
    Console.WriteLine("A'lo");
else if (ball >= 70)
    Console.WriteLine("Yaxshi");
else
    Console.WriteLine("Qoniqarli");
```

► NATIJA (Console chiqishi):

Yaxshi

ball=75 birinchi shartga (≥ 90) mos kelmadi, ikkinchisiga (≥ 70) mos keldi -- shu blok bajarildi, qolganlari tekshirilmadi.

Oddiy va qo'shma operator orasidagi farq nima?

- A** Bitta buyruq -- oddiy operator, { } ichidagi bir nechta buyruq -- qo'shma operator
- B** Ikkalasi ham bir xil narsa, farqi yo'q
- C** Oddiy operator faqat sikllarda, qo'shma operator faqat shartlarda ishlatiladi
- D** Qo'shma operator hech qachon { } talab qilmaydi

✓ JAVOB #1

Oddiy va qo'shma operator orasidagi farq nima?



Bitta buyruq -- oddiy operator, { } ichidagi bir nechta buyruq -- qo'shma operator

B

Ikkalasi ham bir xil narsa, farqi yo'q

C

Oddiy operator faqat sikllarda, qo'shma operator faqat shartlarda ishlatiladi

D

Qo'shma operator hech qachon { } talab qilmaydi



Izoh: Oddiy operator -- yagona bitta buyruq. Qo'shma operator (blok) -- figurali qavslar { } ichiga olingan, bitta butunlik sifatida bajariladigan bir nechta buyruqlar to'plami.

Tanlash operatori: switch-case

- switch-case — bitta o'zgaruvchining KO'P xil qiymatini ketma-ket if-else if yozishdan ko'ra QULAYROQ tekshirish usuli.
- Sintaksisi: `switch (o'zgaruvchi) { case qiymat1: ...; break; case qiymat2: ...; break; default: ...; }`
- Har bir case dan keyin `break;` yozish SHART — aks holda dastur KEYINGI case blokiga ham "quyilib" o'tadi (fall-through xatosi).
- default — hech qaysi case mos kelmasa bajariladigan blok (if-else dagi oxirgi else ga o'xshaydi).

💡 *Amaliy mashg'ulotda switch-case ustida TA bilan ko'proq mashq qilasiz -- bu yerda faqat tanishib o'tamiz.*

Misol: switch-case

```
int kun = 3;

switch (kun)
{
    case 1: Console.WriteLine("Dushanba"); break;
    case 2: Console.WriteLine("Seshanba"); break;
    case 3: Console.WriteLine("Chorshanba"); break;
    default: Console.WriteLine("Noma'lum kun"); break;
}
```

► NATIJA (Console chiqishi):

Chorshanba

switch ichida har bir case dan keyin break; yozishning sababi nima?

- A** Ixtiyoriy, yozmasa ham bo'ladi, hech narsaga ta'sir qilmaydi
- B** Shart, aks holda dastur keyingi case blokiga ham davom etib ketadi
- C** break faqat sikllarda ishlatiladi, switch ichida ma'nosiz
- D** break yozilsa, switch operatori umuman ishlamay qoladi

✓ JAVOB #2

switch ichida har bir case dan keyin break; yozishning sababi nima?

A Ixtiyoriy, yozmasa ham bo'ladi, hech narsaga ta'sir qilmaydi

✓ **Shart, aks holda dastur keyingi case blokiga ham davom etib ketadi**

C break faqat sikllarda ishlatiladi, switch ichida ma'nosiz

D break yozilsa, switch operatori umuman ishlamay qoladi

💡 Izoh: break bo'lmasa, dastur mos kelgan case bajarilgandan keyin KEYINGI case blokiga ham "quyilib" o'tadi (fall-through) -- bu odatda kutilmagan xato natija beradi.



2-BO'LIM

Takrorlash operatori: for

Bir xil amalni bir necha marta qaytarish uchun ishlatiladigan sikllar bilan tanishamiz.

Sikl (loop) nega kerak?

- Agar bir xil amalni 100 marta bajarish kerak bo'lsa, `Console.WriteLine()` ni 100 marta qo'lda yozish AQLga to'g'ri kelmaydi.
- Sikl (loop, takrorlash operatori) — ma'lum shart bajarilguncha bir yoki bir nechta operatorni QAYTA-QAYTA bajarish imkonini beradi.
- C# da uchta asosiy sikl mavjud: `for`, `while`, `do-while` — bugun barchasi bilan tanishamiz.

for operatori sintaksisi

- Sintaksisi: `for (boshlang'ich; shart; qadam) { // takrorlanadigan kod }`
- BOSHLANG'ICH — sikl SONCHI o'zgaruvchisi bir marta e'lon qilinadi (masalan `int i = 1`).
- SHART — har bir aylanishdan OLDIN tekshiriladi; TRUE bo'lsa sikl davom etadi, FALSE bo'lsa to'xtaydi.
- QADAM — har bir aylanish TUGAGANDAN keyin bajariladi (odatda `i++` yoki `i--`).
- `for` — takrorlanish soni OLDINDAN ma'lum bo'lgan holatlar uchun eng qulay tanlov.

Misol: 1 dan 5 gacha chiqarish

```
for (int i = 1; i <= 5; i++)  
{  
    Console.WriteLine(i);  
}
```

► NATIJA (Console chiqishi):

```
1  
2  
3  
4  
5
```

Misol: 1 dan 10 gacha sonlar yig'indisi

```
int yigindi = 0;

for (int i = 1; i <= 10; i++)
{
    yigindi += i;
}

Console.WriteLine(yigindi);
```

► NATIJA (Console chiqishi):

55

yigindi — «akkumulyator» o'zgaruvchi: har aylanishda joriy i qiymati unga qo'shib boriladi.



VIKTORINA #3

for (int i = 1; i <= 5; i++) siklida i qanday qiymatlarni oladi?

A 1, 2, 3, 4

B 1, 2, 3, 4, 5

C 0, 1, 2, 3, 4

D Cheksiz aylanadi, hech qachon to'xtamaydi

O'ylab ko'ring... javobingizni chatga yozing!

✓ JAVOB #3

for (int i = 1; i <= 5; i++) siklida i qanday qiymatlarni oladi?

A 1, 2, 3, 4

✓ 1, 2, 3, 4, 5

C 0, 1, 2, 3, 4

D Cheksiz aylanadi, hech qachon to'xtamaydi

💡 Izoh: $i = 1$ dan boshlanadi, har safar 1 ga oshadi, va $i \leq 5$ sharti FALSE bo'lguncha (ya'ni $i = 6$ bo'lganda) davom etadi -- shuning uchun 1, 2, 3, 4, 5 qiymatlarini oladi.



III

3-BO'LIM

Takrorlash operatorlari: **while, do-while**

Takrorlanish soni OLDINDAN noma'lum bo'lganda ishlatiladigan sikllar.

while operatori

- Sintaksisi: `while (shart) { // shart true bo'lgan ekan takrorlanadi }`
- Shart HAR BIR aylanishdan OLDIN tekshiriladi — agar boshidanoq FALSE bo'lsa, sikl BIR MARTA HAM bajarilmaydi.
- `while` — takrorlanish soni OLDINDAN ma'lum bo'lmagan, balki biror shartga bog'liq holatlar uchun qulay (masalan foydalanuvchi to'g'ri son kiritmaguncha so'rash).
- MUHIM: sikl ichida shartni FALSE qiladigan kod bo'lishi shart, aks holda CHEKSIZ SIKL (infinite loop) hosil bo'ladi!

Misol: while sikli

```
int i = 1;

while (i <= 5)
{
    Console.WriteLine(i);
    i++; // bu qatorsiz sikl CHEKSIZ bo'lardi!
}
```

► NATIJA (Console chiqishi):

```
1
2
3
4
5
```

do-while operatori

- Sintaksisi: `do { // kod } while (shart);` (diqqat: oxirida ";" qo'yiladi!)
- while dan FARQI: shart AYLANISH TUGAGANDAN KEYIN tekshiriladi — shuning uchun kod blok KAMIDA BIR MARTA bajariladi, shart boshidanoq FALSE bo'lsa ham.
- Foydalanuvchidan kirish so'rash kabi "kamida bir marta bajarilishi kerak" vaziyatlarda juda qulay.

Misol: do-while -- shart FALSE bo'lsa ham bir marta ishlaydi

```
int son = 10;

do
{
    Console.WriteLine("Bajarildi!");
} while (son < 5);
```

► NATIJA (Console chiqishi):

Bajarildi!

son=10 bo'lgani uchun shart (son<5) boshidanoq FALSE, lekin do-while baribir BIR MARTA bajardi -- while bo'lganda bu qator umuman chiqmasdi.

for, while, do-while -- qaysi birini qachon tanlash

Sikl	Qachon ishlatiladi	Kamida 1 marta ishlaydimi?
for	Takrorlanish soni OLDINDAN aniq ma'lum bo'lsa	Yo'q (shart false bo'lsa umuman ishlamaydi)
while	Takrorlanish soni shartga bog'liq, oldindan noma'lum	Yo'q
do-while	Kod KAMIDA bir marta bajarilishi shart bo'lsa	Ha, har doim kamida bir marta

break va continue kalit so'zlari

- `break`; — siklni DARHOL to'xtatib, undan chiqib ketadi (switch ichida ham xuddi shu vazifani bajarishini ko'rgan edik).
- `continue`; — joriy aylanishni DARHOL tugatib, siklning KEYINGI aylanishiga o'tadi (siklni butunlay to'xtatmaydi).
- Ikkalasi ham ko'pincha `if` operatori bilan birga ishlatiladi — masalan "agar son 5 ga teng bo'lsa, siklni to'xtat".



while va do-while orasidagi ASOSIY farq nima?

- A Ikkalasi ham bir xil, farqi yo'q
- B while shartni AVVAL tekshiradi, do-while esa AVVAL bajarib, keyin shartni tekshiradi -- shuning uchun do-while kamida bir marta ishlaydi
- C do-while har doim cheksiz aylanadi
- D while faqat sonlar bilan, do-while faqat satrlar bilan ishlaydi

✓ JAVOB #4

while va do-while orasidagi ASOSIY farq nima?

A Ikkalasi ham bir xil, farqi yo'q

✓ **while shartni AVVAL tekshiradi, do-while esa AVVAL bajarib, keyin shartni tekshiradi -- shuning uchun do-while kamida bir marta ishlaydi**

C do-while har doim cheksiz aylanadi

D while faqat sonlar bilan, do-while faqat satrlar bilan ishlaydi

💡 Izoh: while shartni har aylanishdan OLDIN tekshiradi (false bo'lsa umuman ishlamasligi mumkin), do-while esa avval kodni bir marta bajarib, SO'NG shartni tekshiradi.

Cheksiz sikl (infinite loop) xatosi

- Agar sikl sharti HECH QACHON FALSE bo'lmasa, dastur ABADIY aylanaveradi va "qotib qoladi" (hang).
- Eng ko'p uchraydigan sabab — while yoki for ichida sikl o'zgaruvchisini o'zgartirishni (masalan `i++`;) UNUTIB QOLDIRISH.
- Agar dastur to'satdan javob bermay qolsa (konsolga hech narsa chiqmay, tugamay tursa) — bu ko'pincha aynan shu xato.
- Yechim: har doim sikl ICHIDA shartni oxir-oqibat FALSE qiladigan kod (masalan hisoblagichni oshirish) borligiga ishonch hosil qiling.

 *Visual Studio'da qotib qolgan dasturni to'xtatish uchun konsol oynasida Ctrl+C bosish yoki "Stop" tugmasini bosish.*

Yangi boshlovchilar ko'p qiladigan xatolar

- if shartidan keyin ; (nuqta-vergul) qo'yib yuborish: `if (yosh >= 18); { ... }` -- bu shart bilan blokni AJRATIB QO'YADI, blok har doim bajariladi!
- for siklida qadam (`i++`) ni yozishni unutish yoki noto'g'ri yozish -- cheksiz siklga olib kelishi mumkin.
- do-while operatorining oxirida ; (nuqta-vergul) qo'yishni unutish -- bu SHART, aks holda kompilyatsiya xatosi.
- switch ichida `break;` ni unutib qoldirish -- fall-through xatosiga olib keladi.
- `==` o'rniga `=` yozib, shartni chalkashtirib yuborish (o'tgan darsda ko'rgan edik).

Mustaqil bajarish uchun topshiriq

- 1-topshiriq: for sikli yordamida 1 dan 20 gacha bo'lgan JUFT sonlarni ekranga chiqaring (% amalidan foydalaning).
- 2-topshiriq: while sikli yordamida 100 dan 1 gacha KAMAYIB boruvchi sonlarni chiqaring.
- 3-topshiriq: do-while yordamida foydalanuvchidan "chiqish uchun 0 kiriting" deb so'rovchi oddiy dastur yozing.
- 4-topshiriq (qo'shimcha): if-else if-else zanjiri yordamida kiritilgan baholash balliga (100-ballik tizim) mos 5-baholik baho (2,3,4,5) chiqaruvchi dastur yozing.

 *Har bir siklni yozishdan oldin, avval sikl NECHA MARTA aylanishi kerakligini qog'ozda hisoblab ko'ring!*

Bugungi dars xulosasi

- Oddiy operator — bitta buyruq; qo'shma operator (blok) — { } ichidagi bir nechta buyruq.
- if / if-else / else if — shartga qarab turli kod bajarish uchun ishlatiladi.
- switch-case — bitta o'zgaruvchining ko'p qiymatini tekshirishda if-else zanjiridan qulayroq; har case dan keyin break; SHART.
- for — takrorlanish soni OLDINDAN ma'lum bo'lganda; while — shartga bog'liq, oldindan noma'lum bo'lganda.
- do-while — kod KAMIDA bir marta bajarilishi kerak bo'lganda; shart oxirida tekshiriladi.
- break — siklni to'xtatadi; continue — joriy aylanishni o'tkazib yuboradi. Sikl shartini har doim FALSE bo'la oladigan qilib yozing!

5-Ma'ruza: Massivlar

- Massiv (array) tushunchasi va uni e'lon qilish.
- Bir o'lchamli massivlar bilan ishlash.
- Ko'p o'lchamli massivlar bilan tanishuv.
- foreach operatori va massivni tasodifiy son bilan to'ldirish.
- Tayyorgarlik: bugungi sikl mashqlarini bajarib kelishni unutmang!

Foydali adabiyotlar va havolalar

Darsliklar

- Adambayev U.E. va b. "Algoritmik tillar va dasturlash. C# tilida dasturlash asoslari" (2023)
- Mo'minov B.B. "Dasturlash 1" (2021)
- Albahari B. "C# 7.0. Справочник" (2018)

Onlayn manbalar

- metanit.com/sharp/tutorial — Operatorlar va sikllar bo'limi
- learn.microsoft.com/dotnet/csharp/language-reference/statements
- www.dastur.uz — o'zbek tilida dasturlash sayti

Rahmat! Savollaringiz bormi?

-  Aloqa uchun: (o'qituvchi email/telefon — dars vaqtida beriladi)
-  Amaliy topshiriq va mashg'ulotlar — TA bilan davom etadi
-  Keyingi ma'ruza: Massivlar
-  Faol qatnashganingiz uchun rahmat!